🎬 GSAP — GreenSock Animation Platform

Personal Notes & Documentation

> **This is my personal learning notebook for GSAP.** It is also shared with students as a reference guide. Everything here is explained step-by-step, with working code examples, so anyone — beginner or intermediate — can follow along easily.

🗂 Folder Structure

```
gsap/
│
├── 01-Basics/
│   ├── 01-to-and-from/
│   │   ├── index.html
│   │   ├── style.css
│   │   └── script.js
│   │
│   ├── 02-stagger/
│   │   ├── index.html
│   │   ├── style.css
│   │   └── script.js
│   │
│   └── 03-timeline/
│       ├── index.html
│       ├── style.css
│       └── script.js
│
└── README.md   ← You are here
```

📌 Table of Contents

- 🎬 GSAP — GreenSock Animation Platform
  - Personal Notes & Documentation
  - 🗂 Folder Structure
  - 📌 Table of Contents
  - What is GSAP?
  - How to Include GSAP
  - gsap.to()
    - Syntax for `to()`
    - Example — Basic Animation (`to`)
      - `index.html`
      - `style.css`
      - `script.js`

        - Special Properties Explained (`to`)
            - `x` and `y`
            - `duration`
            - `delay`
            - `backgroundColor`
            - `scale`
            - `rotate`
            - `repeat`
            - `yoyo`
            - `ease`
            - `onComplete`
            - `paused`
            - Controlling a Tween with a Variable
    - gsap.from()
        - Syntax for `from()`
        - Example — Basic Animation (`from`)
            - `script.js`
    - gsap.to() vs gsap.from()
    - Stagger
        - The Problem Without Stagger
            - `script.js` — ❌ Repetitive way (without stagger)
        - The Solution — Using Stagger
            - `script.js` — ☑ Clean way (with stagger)
            - How `stagger` works visually:
            - Advanced stagger (optional — for later)
    - gsap.timeline()
        - What is a Timeline?
        - Why Use Timeline Over Stagger?
        - Syntax for Timeline
        - Example — Sequential Animation
        - The Real Power — Custom Sequence Control
        - Complete File Reference — Timeline
            - `index.html`
            - `style.css`
            - `script.js`
    - Complete File Reference
        - 01-to-and-from/
        - 02-stagger/
        - 03-timeline/
    - Quick Reference Card
        - Tween Properties
        - Timeline Methods

What is GSAP?

GSAP (GreenSock Animation Platform) is one of the most powerful JavaScript animation libraries available. It allows you to animate:

- CSS properties (like `x`, `y`, `opacity`, `backgroundColor`, `scale`, `rotation`)
- DOM elements
- SVG
- Canvas
- Any JavaScript object — literally anything

It is incredibly fast, cross-browser compatible, and gives you fine-grained control over every animation.

How to Include GSAP

You can include GSAP via a CDN (easiest way for beginners) by adding this `<script>` tag **before your own script** inside the `<body>`:

```
<script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>
<script src="script.js"></script>
```

> ⚠ **Order matters!** Always load the GSAP CDN script **before** your own `script.js`, otherwise GSAP won't be available when your code runs.

---

gsap.to()

`gsap.to()` is the **most commonly used** type of animation in GSAP.

- You define **where you want the element to go** (the destination/end values).
- GSAP automatically figures out the **starting values** (the current state of the element).
- Think of it as: "*Animate FROM the current state TO these values.*"

Syntax for `to()`

```
gsap.to(target, vars);
```

Parameter	What it means
`target`	The element(s) you want to animate. Can be a CSS selector like `".box"`, `"#id"`, or a direct DOM reference.
`vars`	An object `{}` containing the properties to animate and special control options like `duration`, `delay`, `ease`, etc.

Example — Basic Animation (`to`)

index.html

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>GSAP Basics — to() and from()</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <!-- This is the box we are going to animate -->
    <div class="box"></div>

    <!-- Load GSAP from CDN first -->
    <script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>

    <!-- Then load our animation script -->
    <script src="script.js"></script>
  </body>
</html>
```

style.css

```css
/* Reset default browser margin and padding */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Make html and body take full screen height */
html,
body {
  height: 100%;
}

/* The box we will animate */
.box {
  height: 200px;
  width: 200px;
  background-color: violet; /* starting color */
}
```

script.js

```
// gsap.to() — Animates FROM current state TO the values you define

gsap.to(".box", {
  x: 300, // Move 300px to the RIGHT along X-axis (translateX)
  y: 400, // Move 400px DOWN along Y-axis (translateY)
  duration: 2, // The animation takes 2 seconds to complete
  delay: 2, // Wait 2 seconds before the animation starts
  backgroundColor: "yellow", // Change background color from violet → yellow
  scale: 1.5, // Scale up the element to 1.5× its original size
  rotate: 360, // Rotate the element 360 degrees

  repeat: 4, // Repeat the animation 4 more times (total = 5 plays)
  // repeat: 0  → plays only once (default)
  // repeat: -1 → repeats forever (infinite loop)

  yoyo: true, // On every alternate repeat, play the animation in REVERSE
  // This creates a back-and-forth effect
});
```

Special Properties Explained (`to`)

These go inside the `vars` object `{}` of your `gsap.to()` call.

`x` and `y`

```
gsap.to(".box", { x: 300, y: 400 });
```

- `x` moves the element **horizontally** (positive = right, negative = left).
- `y` moves the element **vertically** (positive = down, negative = up).
- These are shorthand for `translateX` and `translateY` CSS transforms.
- GSAP uses transforms under the hood, which is **GPU-accelerated** and much better for performance than animating `left` or `top`.

`duration`

```
gsap.to(".box", { x: 300, duration: 2 });
```

- Controls **how long** the animation takes, in **seconds**.
- Default is `0.5` seconds if you don't specify it.

`delay`

```
gsap.to(".box", { x: 300, duration: 2, delay: 2 });
```

- Waits the specified number of **seconds** before starting the animation.
- Useful when you want animations to start at different times.

`backgroundColor`

```
gsap.to(".box", { backgroundColor: "yellow" });
```

- Animates the background color **smoothly** from the current color to the new one.
- Write it in **camelCase** (not `background-color`) because this is JavaScript.

`scale`

```
gsap.to(".box", { scale: 1.5 });
```

- Scales the element up or down.
- `scale: 1` = original size.
- `scale: 1.5` = 50% bigger.
- `scale: 0.5` = half the size.

`rotate`

```
gsap.to(".box", { rotate: 360 });
```

- Rotates the element by the given number of **degrees**.
- `360` = one full clockwise rotation.
- `-360` = one full counter-clockwise rotation.

`repeat`

```
gsap.to(".box", { x: 300, repeat: 4 });
```

- Repeats the animation after it completes.
- `repeat: 1` → plays **2 times** total (1 original + 1 repeat).

- `repeat: 0` → plays **once** (default).
- `repeat: -1` → repeats **infinitely** (never stops).

`yoyo`

```
gsap.to(".box", { x: 300, repeat: -1, yoyo: true });
```

- When `yoyo: true`, every **alternate repeat** plays the animation **backwards**.
- This creates a smooth **back-and-forth / ping-pong** effect.
- Only works when `repeat` is set to a non-zero value.

Without yoyo:

```
→ play → jump back → play → jump back → play ...
```

With yoyo:

```
→ play forward → play backward → play forward → play backward ...
```

`ease`

```
gsap.to(".box", { x: 300, ease: "bounce.out" });
```

- Controls the **rate of change** during the animation (acceleration/deceleration).
- Think of it as the "feel" or "personality" of the animation.
- Common eases:
  - `"power1.out"` → default, gentle deceleration
  - `"elastic"` → springy bounce
  - `"bounce.out"` → bounces at the end
  - `"none"` → perfectly linear (robotic)
  - `"back.in"` → overshoots slightly at the start

`onComplete`

```
gsap.to(".box", {
  x: 300,
  duration: 2,
```

```
  onComplete: function () {
    console.log("Animation is done!");
  },
});
```

- A **callback function** that runs when the animation finishes.
- Great for chaining animations or triggering other code after an animation ends.

`paused`

```
let tween = gsap.to(".box", { x: 300, paused: true });
// Animation won't start automatically

// Start it manually whenever you want:
tween.play();
```

- By default, animations **start immediately** when you call `gsap.to()`.
- Setting `paused: true` stops it from auto-starting.
- You can then control it manually using `.play()`, `.pause()`, `.reverse()`, etc.

Controlling a Tween with a Variable

```
// Store the tween in a variable
let tween = gsap.to(".box", { rotation: 360, duration: 5, ease: "elastic" });

// Now you have full control:
tween.pause(); // Pause the animation
tween.play(); // Resume it
tween.reverse(); // Play it backwards
tween.seek(2); // Jump to the 2-second mark
tween.progress(0.5); // Jump to exactly 50% of the animation
tween.restart(); // Start from the beginning
```

---

gsap.from()

`gsap.from()` is the **opposite** of `gsap.to()`.

- You define **where the element should START** (the starting values).
- GSAP uses the **element's current state** as the destination (end values).
- Think of it as: "*Animate FROM these values TO the current state.*"

> It's like watching a `gsap.to()` play **in reverse**.

Syntax for `from()`

```
gsap.from(target, vars);
```

Same parameters as `gsap.to()` — the difference is only in **direction**.

Example — Basic Animation (`from`)

`script.js`

```
// gsap.from() — Animates FROM these values TO the element's current position

gsap.from(".box", {
  x: 300, // Element starts 300px to the right, slides to original position
  y: 400, // Element starts 400px below, moves up to original position
  duration: 2, // Animation takes 2 seconds
  delay: 2, // Wait 2 seconds before starting
  backgroundColor: "yellow", // Starts as yellow, transitions to its CSS color
(violet)
  scale: 1.5, // Starts at 1.5× size, shrinks to normal size
});
```

Visual difference:

`gsap.to()`	`gsap.from()`
Starts at current position	Starts at the values you define
Ends at the values you define	Ends at current position
`violet box → moves right to x:300`	`box starts at x:300 → moves left to original position`

> ⚠ **Common mistake:** A lot of beginners get confused by `from()` and use it incorrectly. Remember — the values in `from()` are **where the element begins**, not where it goes.

---

gsap.to() vs gsap.from()

```
// TO: box starts at its CSS position, moves 300px to the right
gsap.to(".box", { x: 300, duration: 2 });

// FROM: box starts 300px to the right, moves back to its CSS position
gsap.from(".box", { x: 300, duration: 2 });
```

Think of it like this:

- `to()` → "Go there."
- `from()` → "Come from there."

Stagger

Stagger is a **powerful feature** that lets you animate **multiple elements** with a **time offset** between each one — so they don't all animate at the same exact time.

The Problem Without Stagger

Imagine you have 6 boxes and you want each one to animate with a different delay. Without stagger, you'd have to write **6 separate `gsap.to()` calls** with manually set delays:

`script.js` — ❌ Repetitive way (without stagger)

```
// This works but is repetitive and hard to maintain
gsap.to("#box1", { x: 500, duration: 1, delay: 1, yoyo: true, repeat: -1 });
gsap.to("#box2", { x: 500, duration: 1, delay: 2, yoyo: true, repeat: -1 });
gsap.to("#box3", { x: 500, duration: 1, delay: 3, yoyo: true, repeat: -1 });
gsap.to("#box4", { x: 500, duration: 1, delay: 4, yoyo: true, repeat: -1 });
gsap.to("#box5", { x: 500, duration: 1, delay: 5, yoyo: true, repeat: -1 });
gsap.to("#box6", { x: 500, duration: 1, delay: 6, yoyo: true, repeat: -1 });
```

Problems with this approach:

- Lots of repeated code (not DRY — Don't Repeat Yourself)
- Hard to update (if you change the animation, you update it in 6 places)
- Not scalable (what if you had 50 boxes?)

The Solution — Using Stagger

`script.js` — ☑ Clean way (with stagger)

```
// stagger animates all ".box" elements one after another
// with a 1-second gap between each element's start time

gsap.to(".box", {
  x: 500, // Move each box 500px to the right
  duration: 1, // Each box takes 1 second to animate
  repeat: -1, // Each box repeats forever
  yoyo: true, // Each box bounces back and forth
  stagger: 1, // 1 second gap between each box's start time
  // box1 starts at 0s, box2 at 1s, box3 at 2s, etc.
});
```

This single `gsap.to()` call does **exactly the same thing** as 6 separate calls — but in a clean, scalable, and maintainable way.

How `stagger` works visually:

```
Time →  0s      1s      2s      3s      4s      5s
box1:   ▶ starts
box2:           ▶ starts
box3:                   ▶ starts
box4:                           ▶ starts
box5:                                   ▶ starts
box6:                                           ▶ starts
```

Each box starts `stagger` seconds after the previous one.

Advanced stagger (optional — for later)

You can also pass an object to `stagger` for more control:

```
gsap.to(".box", {
  x: 500,
  duration: 1,
  stagger: {
    amount: 2, // Total time to stagger all elements over (2 seconds total)
    from: "end", // Start staggering from the last element
    // from: "center" → start from middle
    // from: "random" → randomize order
  },
});
```

---

gsap.timeline()

What is a Timeline?

A **GSAP Timeline** (`gsap.timeline()`) is a **sequencing tool** — think of it as a container that holds multiple tweens and plays them one after another automatically.

- It acts as a **parent container** for tweens and even other timelines.
- Animations inside a timeline are played **in the order they are listed** in the code.
- You **don't need to manually calculate delays** — the timeline handles timing for you.
- You can add, remove, or reorder animations without breaking the whole sequence.
- Timelines make it possible to create **adjustable and resilient animation sequences**.

> With plain `gsap.to()` calls, you'd have to carefully calculate `delay` values for each animation to sequence them. A timeline eliminates that completely.

Why Use Timeline Over Stagger?

`stagger` is great when you want to animate **the same set of elements** with the **same animation** offset by time.

But `gsap.timeline()` gives you **full control over sequence** — you can:

- Animate **different elements** in any order you want
- Apply **different animations** to each element
- Control **which element goes first, second, third** regardless of DOM position
- Easily **reorder** animations just by moving lines of code

	Stagger	Timeline
Multiple elements	☑	☑
Same animation on all	☑	☑
Custom order	✕	☑
Different animations per element	✕	☑
No manual delay math	☑	☑

Syntax for Timeline

```
// Step 1: Create the timeline
const tl = gsap.timeline();

// Step 2: Add tweens onto it — they play in the order written
tl.to(target, vars);
tl.to(target, vars);
tl.to(target, vars);
// Each tween plays AFTER the previous one finishes automatically
```

You can also pass options to the timeline itself:

```
const tl = gsap.timeline({
  repeat: -1, // Repeat the entire sequence forever
  yoyo: true, // Play the entire sequence in reverse on alternate repeats
  delay: 1, // Wait 1 second before the sequence starts
});
```

Example — Sequential Animation

```
const tl = gsap.timeline();

// These play one after another — no delay math needed!
tl.to(".box", { x: 800, duration: 1 });
tl.to(".box", { y: 400, duration: 1 });
tl.to(".box", { rotate: 360, duration: 1 });
```

Timeline handles the sequencing automatically:

```
0s → 1s : box moves right (x: 800)
1s → 2s : box moves down (y: 400)
2s → 3s : box rotates 360°
```

Without timeline, you'd have to write:

```
// Without timeline — manual delay calculation is error-prone
gsap.to(".box", { x: 800, duration: 1, delay: 0 });
gsap.to(".box", { y: 400, duration: 1, delay: 1 }); // manually calculated
gsap.to(".box", { rotate: 360, duration: 1, delay: 2 }); // manually calculated
```

The Real Power — Custom Sequence Control

This is where timeline **truly shines** over stagger. You can animate elements in **any custom order**, not just top-to-bottom DOM order.

Here's the key example from this lesson — instead of animating box1 → box2 → box3 → box4 → box5 → box6 in order, we animate **odd boxes first (1, 3, 5)** then **even boxes (2, 4, 6)**:

```
const tl = gsap.timeline();

// Animate odd-numbered boxes first
tl.to("#box1", { x: 500, duration: 2 });
tl.to("#box3", { x: 500, duration: 2 });
tl.to("#box5", { x: 500, duration: 2 });

// Then animate even-numbered boxes
tl.to("#box2", { x: 500, duration: 2 });
tl.to("#box4", { x: 500, duration: 2 });
tl.to("#box6", { x: 500, duration: 2 });
```

You simply cannot do this with stagger. Stagger always animates elements in DOM order. Timeline lets you define any sequence you want — just write the tweens in the order you want them to play.

Sequence visualization:

```
0s  → 2s  : box1 moves right  ← odd
2s  → 4s  : box3 moves right  ← odd
4s  → 6s  : box5 moves right  ← odd
6s  → 8s  : box2 moves right  ← even
8s  → 10s : box4 moves right  ← even
10s → 12s : box6 moves right  ← even
```

Complete File Reference — Timeline

index.html

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>GSAP — Timeline</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <!-- 6 boxes — each has its own id for precise timeline control -->
    <div id="box1" class="box"></div>
    <div id="box2" class="box"></div>
    <div id="box3" class="box"></div>
    <div id="box4" class="box"></div>
    <div id="box5" class="box"></div>
    <div id="box6" class="box"></div>

    <!-- GSAP CDN — must come before script.js -->
    <script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>
    <script src="script.js"></script>
  </body>
</html>
```

style.css

```css
* {
  margin: 0;
  padding: 0;
```

```
  box-sizing: border-box;
}

html,
body {
  height: 100%;
  background-color: gray;
}

.box {
  height: 100px;
  width: 100px;
  background-color: violet;
  margin: 10px;
}
```

script.js

```
// ============================================================
// gsap.timeline() — sequence animations one after another
// ============================================================

// Create the timeline instance
// All tweens added to 'tl' will play in the order they are written
const tl = gsap.timeline();

// ============================================================
// BASIC USE — animate a single element in multiple steps
// (uncomment to try)
// ============================================================

// tl.to('.box', {
//   x: 800,
//   duration: 1,
// });

// ============================================================
// WITHOUT TIMELINE'S REAL BENEFIT
// (animating in DOM order — same as stagger would do)
// ============================================================

/*
tl.to("#box1", { x: 500, duration: 2 }); // plays at 0s
tl.to("#box2", { x: 500, duration: 2 }); // plays at 2s (auto-sequenced)
tl.to("#box3", { x: 500, duration: 2 }); // plays at 4s
tl.to("#box4", { x: 500, duration: 2 }); // plays at 6s
tl.to("#box5", { x: 500, duration: 2 }); // plays at 8s
tl.to("#box6", { x: 500, duration: 2 }); // plays at 10s
*/
```

```
// ============================================================
// THE REAL POWER OF TIMELINE — custom animation order ☑
// Stagger always goes in DOM order (box1 → box2 → box3...)
// Timeline lets YOU decide the order
// Here: odd boxes first, then even boxes
// ============================================================

// Odd boxes first (1 → 3 → 5)
tl.to("#box1", { x: 500, duration: 2 }); // starts at 0s
tl.to("#box3", { x: 500, duration: 2 }); // starts at 2s
tl.to("#box5", { x: 500, duration: 2 }); // starts at 4s

// Then even boxes (2 → 4 → 6)
tl.to("#box2", { x: 500, duration: 2 }); // starts at 6s
tl.to("#box4", { x: 500, duration: 2 }); // starts at 8s
tl.to("#box6", { x: 500, duration: 2 }); // starts at 10s
```

Complete File Reference

🗁 01-to-and-from/

index.html

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>GSAP — to() and from()</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <!-- Single box to demonstrate to() and from() -->
    <div class="box"></div>

    <!-- GSAP CDN — must come before script.js -->
    <script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>
    <script src="script.js"></script>
  </body>
</html>
```

style.css

```css
* {
  margin: 0;
  padding: 0;
```

```
  box-sizing: border-box;
}

html,
body {
  height: 100%;
}

.box {
  height: 200px;
  width: 200px;
  background-color: violet;
}
```

script.js

```
// ============================================================
// gsap.to() — animates FROM current state TO these values
// ============================================================

gsap.to(".box", {
  x: 300, // Move 300px to the right (translateX)
  y: 400, // Move 400px down (translateY)
  duration: 2, // 2 seconds for the animation
  delay: 2, // Wait 2 seconds before starting
  backgroundColor: "yellow", // Smoothly change color from violet to yellow
  scale: 1.5, // Scale up to 1.5x the original size
  rotate: 360, // Rotate 360 degrees clockwise

  repeat: 4, // Repeat 4 more times (5 total plays)
  // repeat: 0  → plays once (default)
  // repeat: -1 → repeats infinitely

  yoyo: true, // Alternate repeats play in reverse (back-and-forth)
});

// ============================================================
// gsap.from() — animates FROM these values TO current state
// ============================================================

// gsap.from(".box", {
//     x: 300,                    // Starts 300px to the right, slides to
original position
//     y: 400,                    // Starts 400px below, moves up to original
position
//     duration: 2,               // 2 seconds for the animation
//     delay: 2,                  // Wait 2 seconds before starting
//     backgroundColor: "yellow", // Starts yellow, transitions to violet
//     scale: 1.5,                // Starts 1.5x big, shrinks to normal size
// });
```

02-stagger/

index.html

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>GSAP — Stagger</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <!-- 6 boxes — all share the class "box" -->
    <!-- GSAP will target all of them at once using ".box" -->
    <div id="box1" class="box"></div>
    <div id="box2" class="box"></div>
    <div id="box3" class="box"></div>
    <div id="box4" class="box"></div>
    <div id="box5" class="box"></div>
    <div id="box6" class="box"></div>

    <!-- GSAP CDN — must come before script.js -->
    <script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js">
</script>
    <script src="script.js"></script>
  </body>
</html>
```

style.css

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html,
body {
  height: 100%;
  background-color: gray;
}

.box {
  height: 100px;
  width: 100px;
  background-color: violet;
```

```
    margin: 10px;
  }
```

script.js

```
// ============================================================
// WITHOUT STAGGER — repetitive, hard to maintain
// ============================================================

/*
gsap.to("#box1", { x: 500, duration: 1, delay: 1, yoyo: true, repeat: -1 });
gsap.to("#box2", { x: 500, duration: 1, delay: 2, yoyo: true, repeat: -1 });
gsap.to("#box3", { x: 500, duration: 1, delay: 3, yoyo: true, repeat: -1 });
gsap.to("#box4", { x: 500, duration: 1, delay: 4, yoyo: true, repeat: -1 });
gsap.to("#box5", { x: 500, duration: 1, delay: 5, yoyo: true, repeat: -1 });
gsap.to("#box6", { x: 500, duration: 1, delay: 6, yoyo: true, repeat: -1 });
*/

// ============================================================
// WITH STAGGER — clean, DRY, and scalable ☑
// ============================================================

gsap.to(".box", {
  x: 500, // Move all boxes 500px to the right
  duration: 1, // Each box takes 1 second to animate
  repeat: -1, // Each box repeats forever
  yoyo: true, // Each box bounces back and forth
  stagger: 1, // 1 second gap between each box starting its animation
  // box1 → 0s, box2 → 1s, box3 → 2s, box4 → 3s, box5 → 4s, box6 → 5s
});
```

🗁 03-timeline/

(Files are shown in the Timeline File Reference section above)

📝 Quick Reference Card

Tween Properties

Property	Type	Default	What it does
`x`	number	—	Move horizontally (px)
`y`	number	—	Move vertically (px)
`duration`	number	`0.5`	Animation length in seconds
`delay`	number	`0`	Wait before starting (seconds)

Property	Type	Default	What it does
`ease`	string	`"power1.out"`	Animation feel/curve
`repeat`	number	`0`	Extra repeats (−1 = infinite)
`yoyo`	boolean	`false`	Back-and-forth on repeat
`stagger`	number/object	—	Offset between multiple targets
`scale`	number	—	Scale factor (1 = normal)
`rotate`	number	—	Rotation in degrees
`opacity`	number	—	0 = invisible, 1 = visible
`backgroundColor`	string	—	CSS color value
`paused`	boolean	`false`	Don't auto-start
`onComplete`	function	—	Callback when animation ends

Timeline Methods

Method	What it does
`gsap.timeline()`	Creates a new timeline instance
`tl.to()`	Adds a `to` tween to the sequence
`tl.from()`	Adds a `from` tween to the sequence
`tl.fromTo()`	Adds a `fromTo` tween to the sequence
`tl.pause()`	Pauses the entire timeline
`tl.play()`	Plays/resumes the timeline
`tl.reverse()`	Plays the timeline backwards
`tl.restart()`	Restarts from the beginning

> 📌 **More topics coming soon:** `gsap.fromTo()`, Timeline position parameter (`<`, `>`, labels), ScrollTrigger, and more.